

Algo PBX — Infrastructure Guide 2 of 2

Configuring Credentials, Dinstar & Going Live

Wiring up `/admin/settings`, connecting the physical Dinstar UC2000 gateway over LAN, and cutting your GoDaddy domain over to Cloudflare for free SSL.

Prerequisite: "Deploying Algo PBX on a Linux VM" completed — the stack is up and you have created the first admin account at `/setup`.

The `/setup` and `/admin/settings` screenshots below were captured live against the production standalone build running locally. GoDaddy/Cloudflare/Dinstar screens remain reference diagrams — menu wording varies by account region and firmware version, but the fields and order of operations are accurate.

Contents

1. How `/admin/settings` works
2. Email — Resend
3. WhatsApp — OpenWA
4. SMS & Voice — the Dinstar UC2000 (LAN-connected gateway)
5. Optional: Meta Cloud API & Firebase
6. Domain cutover: GoDaddy → Cloudflare, and free SSL
7. Final checklist
8. Appendix A — `/setup` first-run screenshot
9. Appendix B — Ports on the telephony side

1. How /admin/settings works

Every external-service credential (Resend, OpenWA, Meta, Dinstar, Firebase, CRM webhook secret) is admin-configurable here, encrypted at rest (AES-256-GCM), and takes effect immediately with no restart. The only exceptions are telephony-level settings (`AMI` , `Coturn` , `VM_PUBLIC_DOMAIN`) — those stay in `.env` / `pbx_configs/` by design and need a restart.

Runtime Settings

These take effect immediately — no restart needed. Secret values are never shown again once saved; leaving a field blank keeps the existing value.

EMAIL (RESEND)

Test connection

Resend API Key

https://resend.com/api-keys — used only to deliver agent invite emails.

configured, ending null

Save

Configured (ending null)

Invite From Address

invites@algopbx.local

Save

Configured

WHATSAPP — OPENWA

Test connection

OpenWA Base URL

http://localhost:2785

Save

Configured

OpenWA API Key

configured, ending null

Save

Configured (ending null)

OpenWA Webhook Shared Secret

Must match the value configured on the OpenWA sidecar's outbound webhook.

configured, ending null

Save

Configured (ending null)

WHATSAPP — META CLOUD API (FALLBACK)

Test connection

Meta WABA Access Token

not configured

Save

Meta Phone Number ID

not configured

Save

SMS — DINSTAR UC2000

Test connection

Dinstar Gateway Address

Host or origin reachable over the Tailscale route the voice trunk already uses.

192.168.1.50

Save

Configured

Dinstar Admin Username

admin

Save

Configured

Dinstar Admin Password

configured, ending null

Save

Configured (ending null)

Configured (ending null)

OTP DELIVERY

OTP Delivery Channel
Which channel sends verification codes for agent registration and login 2FA.

OPENWA

Save

Configured

OTP WhatsApp Instance
Which paired Walnstance sends OTPs when the channel is OpenWA — leave blank to use the first available instance. Dedicate one SIM here to isolate OTP traffic from customer messaging.

not configured

Save

Meta OTP Template Name
Only used when OTP_CHANNEL is META_CLOUD — must be a pre-approved authentication-category template.

agent_otp_verification

Save

Configured

Meta OTP Template Language

en

Save

Configured

FIREBASE (OPTIONAL OTP CHANNEL)

Test connection

Firestore API Key
Not a secret — identifies the Firebase project, does not authorize anything.

not configured

Save

Firestore Auth Domain

not configured

Save

Firestore Project ID

not configured

Save

Firestore Service Account (JSON)
The full service-account keyfile contents. This IS the actual secret — never sent to the browser.

not configured

Save

CRM WEBHOOKS

CRM Webhook Signing Secret
Shared HMAC secret used when a WebhookSubscription row has no per-subscription secret of its own.

configured, ending null

Save

Configured (ending null)

Live screenshot: /admin/settings showing Email (Resend), WhatsApp (OpenWA), SMS (Dinstar UC2000), OTP, Firebase and CRM Webhooks sections. Captured on an earlier build — the current app wraps every admin page in the grouped-sidebar shell with a live system-health pill; fields and layout shown here are unchanged.

Each section has a **Test connection** button — always click it after saving. It is the fastest way to catch a typo'd key or an unreachable host before an agent hits the same failure live.

2. Email — Resend

Required before any agent invite can be sent.

1. Sign up / log in at resend.com.
2. Add and verify your sending domain under **Domains** (adds a few DNS records — see Section 6 if you are also moving DNS to Cloudflare; do this after the cutover so you only touch DNS once).
3. Go to **API Keys** → **Create API Key**, copy the key (shown once).
4. In Algo PBX: `/admin/settings` → `Email`, paste the key and the verified sender address, then click **Test connection**.

3. WhatsApp — OpenWA (four SIM ports)

The Dinstar holds up to four GSM SIMs, and each SIM can carry its own WhatsApp number. Algo PBX runs **one OpenWA engine** that hosts all of them as separate sessions — so there are exactly two layers to configure, and only one of them lives in settings:

Layer	Where	How many
OpenWA connection (sidecar host, API key, webhook secret)	<code>/admin/settings</code> → WhatsApp: OpenWA	1 — configured once, shared by every number
WhatsApp numbers (sessions)	<code>/admin/whatsapp</code> — the SIM port board	up to 4 — one per GSM port

1. Configure the OpenWA connection details (host/webhook secret) at `/admin/settings` → WhatsApp: OpenWA, then click **Test connection**. This is a one-time step.
2. Open `/admin/whatsapp`. You get a fixed board of four slots — **SIM Port 1–4**, matching the gateway's physical ports — all live at once.
3. For each port you're using: type the SIM's label into its slot and click **Start pairing**. The slot immediately asks for that number and shows a big pairing code (tap to copy) — on the phone, go to WhatsApp → Linked Devices → Link with phone number instead, enter the code, done. Prefer scanning? Toggle *Scan QR instead* in the same slot.
4. All four slots keep their codes visible simultaneously — you can link four phones back-to-back without navigating anywhere.
5. Each slot flips to green **Connected** as its phone links; the header counts `n/4 connected`. Don't rely on OTP delivery until the port you expect to send OTPs shows Connected.

Screenshot pending. This screen was redesigned after the last capture was taken. Recapture it on the live VM during go-live — exact steps are documented at the top of `scripts/build-docs.py`.

Tip: any one port can be dedicated to OTP traffic only (isolating its ban-risk from customer conversations) via `OTP_WA_INSTANCE_ID` in `.env`. Ports map to agents through the SIM-port field when provisioning users in `/admin/users`.

4. SMS & Voice — the Dinstar UC2000

The Dinstar UC2000 is a 4-port GSM gateway that bridges physical SIM cards to Algo PBX two separate ways: a SIP trunk for voice (Asterisk / pjsip.conf) and an HTTP/JSON API for SMS (/admin/settings). Same box, same LAN IP, two integration points — don't confuse them.

4.1 — Physically connect it

1. Connect the Dinstar's LAN port to your office router/switch with a standard Ethernet cable (or directly to the PC's NIC if point-to-point — but a router/switch is strongly preferred so both the Dinstar and the office PC/VM sit on the same routable subnet).
2. Insert your GSM SIM card(s) into the port(s) you intend to use (1–4).
3. Power it on — wait for the LAN/status LEDs to settle (roughly 60–90 seconds for full boot and SIM registration).

Finding its IP address

By default most Dinstar UC2000 units either DHCP an address from your router or ship with a static factory default (commonly 192.168.1.101 or similar — check the label). Two reliable ways to find it:

```
# Option A – router DHCP client list (usually http://192.168.1.1 or http://192.168.0.1)
# look for a device named "UC2000" or a Dinstar MAC OUI.

# Option B – network scan
nmap -sn 192.168.1.0/24          # Linux/macOS
# or use "Advanced IP Scanner" on Windows
```

4.2 — Log into the Dinstar's own web GUI

1. Open http://<dinstar-ip> in a browser on the same network.
2. Default credentials are typically admin / admin — change this immediately after first login.

What to configure inside the Dinstar GUI

Menu (typical)	What to set	Why
VoIP / SIP Trunk	Add a SIP server entry pointing at your Algo PBX VM's LAN/Tailscale IP, port 5060, transport UDP. Leave registration off if using IP-based trunking (matches this repo's pjsip.conf).	Enables the voice trunk — inbound GSM calls route to Asterisk's from-dinstar context; outbound agent calls go out through the SIMs.
HTTP API / SMS API	Enable the HTTP/JSON API (sometimes labelled "GoIP-compatible API" or "Open API"). Set/confirm the admin username and password used for Basic-auth on the API. Note the API port if separate from the web GUI port.	This is what Algo PBX's SMS inbox polls for send/receive.
Network	Either set a DHCP reservation for this MAC on your router, or assign the Dinstar a static IP here. Pin the IP — if it changes later, every config below breaks silently.	Algo PBX references this IP in three separate places; it must not move.

System → Password	Change the default admin password.	This device is reachable from your whole PBX-side network; don't leave it on defaults.
-------------------	------------------------------------	--

UNVERIFIED AGAINST A LIVE UNIT. This repo's SMS adapter targets Dinstar's documented GoIP-compatible HTTP API — `POST /goip_send_sms.html`, `GET /goip_get_sms.html`, `GET /goip_get_status.html`, HTTP Basic auth. Firmware revisions differ in exact path names and auth style. If **Test connection** fails in `/admin/settings`, check your unit's actual HTTP API manual/firmware version.

4.3 — Where each Dinstar value gets typed into Algo PBX

Dinstar GUI value	Goes into	Path / variable
LAN IP (static/reserved)	<code>/admin/settings</code> → Dinstar Gateway Address, and the voice-side trunk config	<code>DINSTAR_LAN_IP</code> + generated <code>pjsip_dinstar.conf</code>
HTTP API admin username	<code>/admin/settings</code> → Dinstar Admin Username	<code>DINSTAR_SMS_USERNAME</code>
HTTP API admin password	<code>/admin/settings</code> → Dinstar Admin Password (the wizard probes which auth style your firmware uses)	<code>DINSTAR_SMS_USERNAME</code> / <code>DINSTAR_AUTH_STYLE</code>

The `/admin/dinstar setup wizard` does the voice-side wiring for you: it scans the Tailscale/LAN subnet for the gateway, probes its credentials, writes the generated Asterisk trunk config (`pjsip_dinstar.conf`), reloads Asterisk over AMI, and *verifies the reload actually took effect*. You never hand-edit `pjsip.conf` :

```
# In the browser, as an admin:
/admin/dinstar → Discover → probe → Apply
# Then confirm on /admin/system that "Asterisk AMI" and
# "Dinstar gateway reachable" both show green.
```

The `/admin/settings` fields (SMS side) are hot-reloaded, no restart. Only the SIM insertion/PIN, changing the device's factory password, and its own SIP-trunk page remain manual inside the Dinstar GUI — the wizard shows a copy-paste values panel for that last one.

4.4 — If the Dinstar-connected PC isn't on the same LAN as the VM

Use Tailscale subnet routing (from `ALGO_PBX_MASTER_DOC.md` §6.3) when the Dinstar sits in an office network separate from wherever the Algo PBX VM runs:

```
# ON THE OFFICE PC (same LAN as the Dinstar):
echo 'net.ipv4.ip_forward = 1' | sudo tee /etc/sysctl.d/99-tailscale.conf
sudo sysctl -p /etc/sysctl.d/99-tailscale.conf
sudo tailscale up --advertise-routes=192.168.1.0/24
# Then: login.tailscale.com -> Machines -> this device -> Edit Route Settings -> APPROVE the route

# ON THE ALGO PBX VM:
```

```
sudo tailscale up --accept-routes  
ping 192.168.1.50  
curl -I http://192.168.1.50
```

If the office PC and the VM are on the same LAN, skip Tailscale entirely — `DINSTAR_LAN_IP` is just its LAN IP.



5. Optional: Meta Cloud API & Firebase

Only needed if you switch `OTP_CHANNEL` away from `OPENWA` in the OTP section of `/admin/settings`.

- **Meta Cloud API** — create a Meta developer app, add the WhatsApp product, grab the permanent access token and phone number ID from `developers.facebook.com`; paste into `/admin/settings` → WhatsApp: Meta Cloud.
- **Firebase** — create a project at `console.firebase.google.com`, enable Cloud Messaging, generate a service-account key under **Project Settings** → **Service Accounts**, paste the JSON into the Firebase section.

6. Domain cutover: GoDaddy → Cloudflare, free SSL

This section is documentation only — nameserver changes affect live DNS and should be done by you directly in your own GoDaddy/Cloudflare accounts. Diagrams below are reference layouts of the current dashboards.

6.1 — Add your domain to Cloudflare first

1. Sign up / log in at `dash.cloudflare.com`.
2. Add a domain, enter the domain you bought on GoDaddy, choose the **Free** plan.
3. Cloudflare scans existing DNS records and shows you a list to import — review them.
4. Cloudflare shows you two nameservers assigned to your domain (e.g. `ana.ns.cloudflare.com`, `bob.ns.cloudflare.com`) — copy these for GoDaddy.

6.2 — Change nameservers at GoDaddy

1. Log into `godaddy.com` → **My Products**.
2. Find your domain, click **DNS** next to it.
3. Under **Nameservers**, click **Change**, choose **Enter my own nameservers (Custom)**.
4. Paste the two Cloudflare nameservers from 6.1, remove any GoDaddy-default ones, **Save**.

Nameserver changes typically take 30 minutes to a few hours, occasionally up to 24–48 hours, to fully propagate. Cloudflare emails you once it detects the change and activates the zone.

6.3 — DNS records in Cloudflare

Once the zone is active, add the A record pointing at your VM:

Type	Name	Content	Proxy status
A	@ (or your subdomain, e.g. <code>pbx</code>)	Your VM's public IP	DNS only (grey cloud) — see warning below

IMPORTANT — PROXY (ORANGE CLOUD) DOES NOT WORK FOR THIS STACK. Cloudflare's proxy only forwards HTTP/HTTPS traffic on a small set of ports and terminates TLS itself. Algo PBX needs raw UDP 5060 (SIP), UDP 10000–20000 (RTP), UDP 20001–30000 (TURN relay), and WSS on 8089 reaching your VM directly — none of that works through the orange-cloud proxy. Set the record to **DNS ONLY** (grey cloud) so it resolves straight to your VM's IP, and get your TLS certificate a different way.

6.4 — Free SSL: one cert pair serves everything

The stack ships a **Caddy reverse proxy** that terminates HTTPS for the web app on 443, plus Asterisk WSS (8089) and Coturn TLS (5349). All three read the *same* certificate pair from `pbx_configs/keys/`, so you issue one Let's Encrypt certificate and copy it once.

Option	How	Notes
DNS-01 via Cloudflare API (recommended)	<code>certbot certonly --dns-cloudflare</code> with a Zone-DNS-edit API token. No port 80 listener needed;	Works with the grey-cloud (DNS-only) record this guide

	renewal never touches the running stack.	requires anyway.
HTTP-01 standalone (fallback)	<code>sudo docker compose stop caddy → certbot certonly --standalone -d ... → start caddy .</code>	Stop Caddy, not <code>web</code> — <code>web</code> listens on <code>:3000</code> , it was never the port-80 conflict.

```
# One-time issuance (DNS-01 shown):
sudo apt install -y certbot python3-certbot-dns-cloudflare
# create /etc/letsencrypt/cloudflare.ini with dns_cloudflare_api_token = ...
sudo certbot certonly --dns-cloudflare \
  --dns-cloudflare-credentials /etc/letsencrypt/cloudflare.ini -d your-domain.example.com
sudo cp /etc/letsencrypt/live/your-domain.example.com/fullchain.pem pbx_configs/keys/
sudo cp /etc/letsencrypt/live/your-domain.example.com/privkey.pem pbx_configs/keys/

# Renewal (every ~60 days) – the deploy hook MUST restart asterisk, coturn
# AND caddy, or all three keep serving the expired certificate from memory:
0 3 * * 1 certbot renew --quiet --deploy-hook \
  "cp -f /etc/letsencrypt/live/YOUR_DOMAIN/{fullchain.pem,privkey.pem} /opt/algo-pbx/pbx_configs/k
```

6.5 — Point `.env` at the final domain

```
VM_PUBLIC_DOMAIN=your-domain.example.com
AUTH_URL=https://your-domain.example.com
```

```
docker compose down
docker compose up -d
```

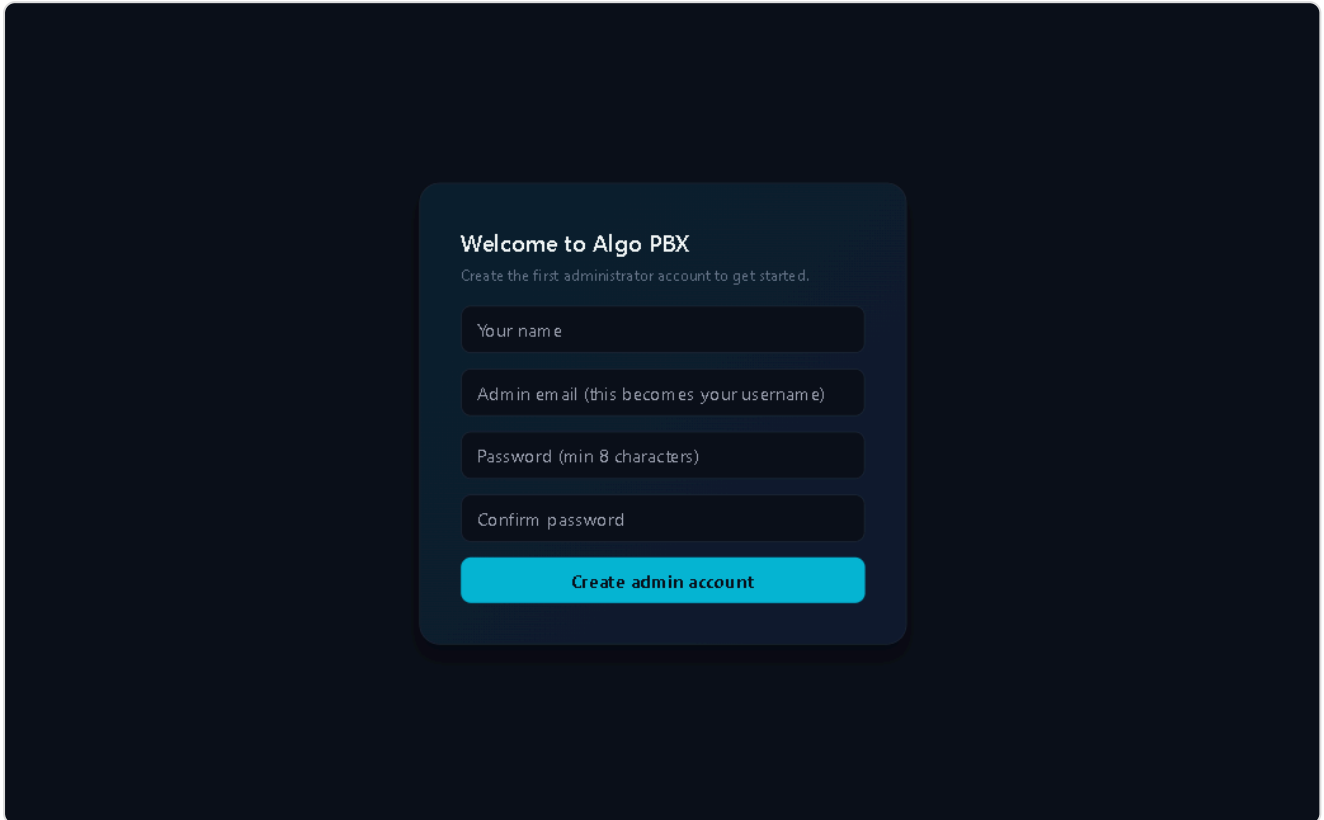
This is one of the few values that lives only in `.env` and requires a restart — it is not editable from `/admin/settings` .

7. Final checklist

- Email (Resend) — connection tested, at least one invite sent successfully
- WhatsApp OpenWA — all four SIM ports paired on the board (each slot green), status Connected on every port in use
- Dinstar setup wizard (`/admin/dinstar`) — discovered, probed, applied; trunk verified by the wizard itself
- Dinstar SIP trunk — outbound test call placed through a SIM, inbound call reaches the ring group
- Dinstar SMS API — `/admin/settings` Test connection passes; a test SMS sent and received
- GoDaddy nameservers pointed at Cloudflare, zone active
- DNS A record — DNS-only (grey cloud), not proxied
- Real TLS cert (Let's Encrypt) installed in `pbx_configs/keys/` — browser trusts `https://` AND `wss://`
- `VM_PUBLIC_DOMAIN` / `AUTH_URL` updated in `.env` , stack restarted
- Certbot renewal cron in place, deploy hook restarting asterisk + coturn + caddy
- Dinstar default admin password changed from factory default
- `/admin/system` readiness page — every check green
- `scripts/setup-firewall.sh` applied; 5060 reachable only over Tailscale
- `scripts/backup.sh` on cron; one restore drill completed

Appendix A — /setup first-run screenshot

When the stack first starts and no admin exists, browse to `/setup` to create the first administrator account. After the first admin is created, `/setup` automatically redirects to `/login`.



Live screenshot: `/setup` rendering the first-administrator creation form in the production standalone build.

Appendix B — Ports on the telephony side

Guide 1 Chapter 2 covers the web-facing ports (443, 80, 8089, TURN/RTP ranges). This appendix covers the one telephony port that behaves differently: **5060/udp**, the SIP signaling path to the Dinstar UC2000.

B.1 — No inbound port is opened anywhere for this

The Dinstar trunk is reached exclusively over the **Tailscale mesh** — never a public port on the UAE office router, and never forwarded through the VM's router (hard constraint, see `LLM.md` §2). Tailscale only needs outbound UDP from the VM to reach its coordination servers; nothing needs to be opened inbound on the router or the VM's firewall for the SIP trunk to work.

B.2 — Verifying it from the VM

```
# Confirm Asterisk is listening for SIP on the Tailscale interface:
sudo ss -ulpn | grep 5060

# Confirm the Dinstar endpoint is actually registered:
docker compose exec asterisk asterisk -rx "pjsip show endpoints"

# Confirm the Tailscale subnet route to the Dinstar is up:
tailscale status
```

B.3 — Diagnosing "call connects but there's no audio"

This is almost always Guide 1 §2.2's RTP range (`10000-20000/udp`), not the SIP port — SIP only carries call setup, the audio itself travels over RTP. If signaling works (the call rings and connects) but there's silence, re-check that the RTP range is actually forwarded and unblocked end-to-end (Guide 1 Chapter 7), not the Tailscale route.

B.4 — Never expose these, regardless of environment

- **5060/udp** — SIP to the Dinstar. Tailscale interface only.
- **5038/tcp** — Asterisk AML. Docker bridge ACL only.
- **5432/tcp** — PostgreSQL. Loopback publish only.